HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

BÁO CÁO ĐỒ ÁN CUỐI KỲ

Sinh viên thực hiện:

B23DCAT021 Trần Việt Anh

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

HỌC KỲ 1 NĂM HỌC 2026-2027

MỤC LỤC

MỤC LỤC 2
DANH MỤC CÁC HÌNH VẼ 4
DANH MỤC CÁC BẢNG BIỂU 4
DANH MỤC CÁC TỪ VIẾT TẮT 5
CHƯƠNG 1. LÝ DO CHỌN ĐỀ TÀI 6
1.1 Bối cảnh và động lực 6
1.2 Vấn đề cần giải quyết 6
1.3 Lý do lựa chọn công nghệ 6
1.4 Mục tiêu đề tài 7
CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG 8
2.1 Tổng quan kiến trúc 8
2.2 Sơ đồ chân kết nối phần cứng 8
2.3 Thiết kế giao thức MQTT 9
2.4 Thiết kế RESTful API 9
2.4.1 API Xác thực (Authentication) 9
2.4.2 API Dữ liệu cảm biến 10
2.4.3 API Lịch sử điều khiển 11
2.4.4 API Thiết bị 13
2.5 Luồng dữ liệu chi tiết 14
2.5.1 Luồng thu thập và hiển thị dữ liệu cảm biến 14
2.5.2 Luồng điều khiển thiết bị hai chiều 15
2.5.3 Luồng đồng bộ trạng thái khi ESP8266 khởi động 15
CHƯƠNG 3. THIẾT KẾ CHI TIẾT 16
3.1 Phân tích yêu cầu và Use Case 16
3.1.1 Các tác nhân trong hệ thống 16
3.1.2 Sơ đồ Use Case tổng quan 16
3.1.3 Đặc tả chi tiết Use Case 16
3.2 Thiết kế cơ sở dữ liệu 20
3.2.1 Mô tả các bảng 20
3.2.2 Quan hệ giữa các bảng 21
3.3 Thiết kế và cài đặt Backend 22
3.3.1 Cấu trúc thư mục dự án 22
3.3.2 Một số điểm kỹ thuật đáng chú ý 23

3.3.3 Danh sách thư viện phụ thuộc 23
3.4 Thiết kế giao diện người dùng 24
CHƯƠNG 4. KẾT QUẢ 25
4.1 Mô hình phần cứng 25
4.2 Kết quả triển khai hệ thống 25
4.3 Giao diện Dashboard – Telemetry Console 25
4.4 Giao diện Tab Data Sensor 26
4.5 Giao diện Tab Action History 26
4.6 Giao diện Tab Developer Profile 27
4.7 Cơ chế Loading và xác nhận trạng thái 27
4.8 Cảnh báo tự động ánh sáng thấp 28
4.9 Kiểm thử API bằng Postman 29
4.10 Đánh giá kết quả 30
CHƯƠNG 5. LỜI KẾT 32
5.1 Tổng kết 32
5.2 Hạn chế hiện tại 32
5.3 Hướng phát triển 33
5.3.1 Mở rộng khả năng giám sát môi trường 33
5.3.2 Phát triển cơ chế điều khiển tự động 33
5.3.3 Xây dựng hệ thống cảnh báo thông minh 33
5.3.4 Tối ưu khả năng lưu trữ và khai thác dữ liệu 33
5.3.5 Ứng dụng Machine Learning trong phân tích và dự đoán 33
TÀI LIỆU THAM KHẢO 35

DANH MỤC CÁC HÌNH VẼ

Hình 1 Sơ đồ kiến trúc tổng quan hệ thống....8
Hình 2 Sơ đồ Use Case tổng quan....16
Hình 3 Sơ đồ ERD....22
Hình 4 Mô hình phần cứng....25
Hình 5 Giao diện Dashboard....26
Hình 6 Giao diện Tab Data Sensor....26
Hình 7 Giao diện Tab Action History....27
Hình 8 Giao diện Tab Developer Profile....27
Hình 9 Trạng thái Loading khi điều khiển thiết bị....28
Hình 10 Toast Notification cảnh báo ánh sáng thấp....29
Hình 11 Kiểm thử API – Đăng nhập lấy JWT Token....29
Hình 12 Kiểm thử API – Điều khiển thiết bị với Bearer Token....30

DANH MỤC CÁC BẢNG BIỂU

Bảng 1 So sánh các giải pháp IoT phổ biến....6
Bảng 2 Công nghệ sử dụng trong hệ thống....7
Bảng 3 Sơ đồ chân kết nối ESP8266....8
Bảng 4 Mô tả các topic MQTT....9
Bảng 5 Mã trạng thái HTTP áp dụng....9
Bảng 6 Bảng các tham số API Dữ liệu cảm biến....10
Bảng 7 Bảng tham số API Lịch sử điều khiển....12
Bảng 8 Mô tả các tác nhân....16
Bảng 9 Đặc tả Use Case: Điều khiển thiết bị....17
Bảng 10 Đặc tả Use Case: Xem dữ liệu cảm biến thời gian thực....17
Bảng 11 Đặc tả Use Case: Xem dữ liệu cảm biến lưu trong CSDL....18
Bảng 12 Đặc tả Use Case: Xem lịch sử điều khiển....19
Bảng 13 Đặc tả Use Case: Đồng bộ trạng thái khi tải lại trang....19
Bảng 14 Mô tả bảng user....20
Bảng 15 Mô tả bảng device....20
Bảng 16 Mô tả bảng sensor....20
Bảng 17 Mô tả bảng sensor_data....21
Bảng 18 Mô tả bảng action_history....21
Bảng 19 Danh sách thư viện phụ thuộc....24

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
IoT	Internet of Things	Internet vạn vật
MQTT	Message Queuing Telemetry Transport	Giao thức truyền tin nhắn nhẹ cho thiết bị IoT
JWT	JSON Web Token	Chuẩn xác thực dựa trên token JSON
API	Application Programming Interface	Giao diện lập trình ứng dụng
REST	Representational State Transfer	Kiến trúc truyền tải trạng thái đại diện
HTTP	HyperText Transfer Protocol	Giao thức truyền siêu văn bản

CHƯƠNG 1. LÝ DO CHỌN ĐỀ TÀI

1.1 Bối cảnh và động lực

Trong bối cảnh cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, Internet of Things (IoT) đã trở thành một trong những nền tảng công nghệ cốt lõi định hình lại cách con người tương tác với môi trường sống. Các hệ thống nhà thông minh (Smart Home) không còn là khái niệm xa lạ mà đang dần trở thành xu hướng phổ biến, mang lại khả năng giám sát, tự động hóa và điều khiển từ xa các thiết bị trong không gian sinh hoạt hàng ngày.

Tuy nhiên, phần lớn các giải pháp IoT thương mại hiện nay hoặc có chi phí cao, hoặc phụ thuộc vào nền tảng đám mây của bên thứ ba mà người dùng không kiểm soát được dữ liệu. Điều này đặt ra bài toán thực tiễn: làm thế nào để xây dựng một hệ thống IoT hoàn chỉnh, tự chủ — từ tầng phần cứng vi điều khiển, tầng giao tiếp giao thức, tầng xử lý dữ liệu backend, đến tầng giao diện người dùng — với chi phí thấp, độ trễ tối thiểu và khả năng mở rộng linh hoạt.

Đây chính là động lực thúc đẩy việc lựa chọn và thực hiện đề tài này.

1.2 Vấn đề cần giải quyết

Một hệ thống giám sát môi trường và điều khiển thiết bị IoT hoàn chỉnh cần đáp ứng đồng thời nhiều yêu cầu kỹ thuật phức tạp:

- Về thời gian thực: Dữ liệu cảm biến cần được cập nhật liên tục và hiển thị tức thì trên giao diện mà không yêu cầu người dùng tải lại trang. Độ trễ từ khi cảm biến đọc giá trị đến khi hiển thị trên Dashboard phải ở mức tối thiểu.
- Về độ tin cậy điều khiển: Lệnh điều khiển thiết bị từ xa cần được xác nhận hai chiều — không thể chỉ gửi lệnh đi mà không biết phần cứng có thực thi thành công hay không. Hệ thống phải tự xử lý được các tình huống phần cứng không phản hồi.
- Về bảo mật: Các thao tác điều khiển thiết bị phải được bảo vệ, ngăn chặn truy cập trái phép từ bên ngoài.
- Về khả năng tra cứu: Toàn bộ dữ liệu cảm biến và lịch sử thao tác cần được lưu trữ có hệ thống, hỗ trợ tìm kiếm và phân trang hiệu quả.

1.3 Lý do lựa chọn công nghệ

Bảng 1 So sánh các giải pháp IoT phổ biến

Tiêu chí	**Giải pháp thương mại**	**Raspberry Pi+Python**	**ESP8266 + Node.js**
Chi phí phần cứng	Cao	Trung bình	Thấp
Tự chủ dữ liệu	Không	Có	Có
Độ trễ realtime	Phụ thuộc Cloud	Trung bình	Thấp
Khả năng mở rộng	Hạn chế	Tốt	Tốt

Tiêu chí	Giải pháp thương mại	Raspberry Pi+Python	ESP8266 + Node.js
Phù hợp học thuật	Không	Có	Có

ESP8266 được chọn vì tích hợp sẵn Wi-Fi, giá thành thấp và hệ sinh thái Arduino phong phú. Node.js phù hợp cho hệ thống I/O-bound như IoT nhờ mô hình bất đồng bộ non-blocking, cho phép xử lý đồng thời MQTT, WebSocket và HTTP mà không tốn nhiều tài nguyên. MQTT được ưu tiên cho giao tiếp phần cứng vì tính nhẹ, độ trễ thấp và hoạt động tốt trên đường truyền không ổn định.

1.4 Mục tiêu đề tài

Đề tài đặt ra các mục tiêu cụ thể sau:

- Giám sát môi trường thời gian thực: Thu thập và hiển thị liên tục các chỉ số nhiệt độ, độ ẩm và cường độ ánh sáng với chu kỳ cập nhật 2 giây, trực quan hóa bằng biểu đồ đường động trên giao diện web.
- Điều khiển thiết bị có xác thực: Cung cấp khả năng bật/tắt relay từ xa qua giao diện web, bảo vệ bằng JWT Authentication, đảm bảo cơ chế phản hồi hai chiều (LOADING → SUCCESS/FAILED) và xử lý timeout tự động.
- Tự động hóa cơ bản: Tự động bật đèn khi cường độ ánh sáng xuống dưới ngưỡng 50 Lux, kèm cảnh báo trực quan.
- Lưu trữ và tra cứu dữ liệu: Ghi nhận toàn bộ dữ liệu cảm biến và lịch sử thao tác vào MySQL, hỗ trợ tìm kiếm, lọc và phân trang.
- Tài liệu hóa API chuẩn: Tích hợp Swagger UI theo chuẩn OpenAPI 3.0, hỗ trợ kiểm thử live các endpoint.

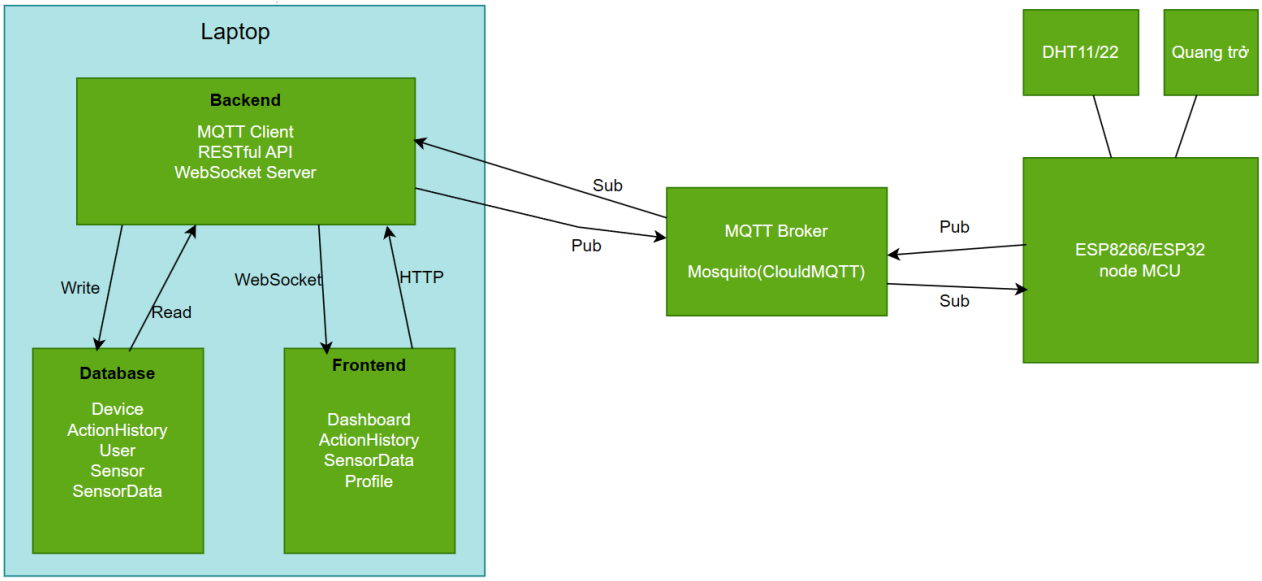
Bảng 2 Công nghệ sử dụng trong hệ thống

Tầng	Công nghệ	Vai trò
Phần cứng	ESP8266, C++/Arduino	Đọc cảm biến, điều khiển relay
Giao thức IoT	MQTT – HiveMQ Broker	Giao tiếp nhẹ phần cứng ↔ server
Bảo mật	JWT (jsonwebtoken)	Xác thực người dùng cho API điều khiển
Backend	Node.js, Express.js	Xử lý logic nghiệp vụ, REST API
Realtime	Socket.io (WebSocket)	Đẩy dữ liệu tức thì server → Web UI
Cơ sở dữ liệu	MySQL	Lưu trữ dữ liệu cảm biến và lịch sử
Frontend	HTML5, CSS3, JavaScript ES6+	Giao diện người dùng
Trực quan hóa	Chart.js	Biểu đồ đường thời gian thực
Tài liệu API	Swagger UI, OpenAPI 3.0	Mô tả và kiểm thử API

CHƯƠNG 2. KIẾN TRÚC HỆ THỐNG

2.1 Tổng quan kiến trúc

Hệ thống được tổ chức theo mô hình ba tầng kết hợp với kiến trúc Publish/Subscribe, sử dụng đồng thời ba giao thức truyền thông độc lập cho ba mục đích khác nhau.



Hình 1 Sơ đồ kiến trúc tổng quan hệ thống

Tầng Phần cứng (Hardware Layer): Vi điều khiển ESP8266 đóng vai trò đầu cuối IoT, đọc dữ liệu từ cảm biến DHT11 (nhiệt độ, độ ẩm) và quang trở LDR (ánh sáng) qua chân GPIO, đồng thời điều khiển các LED/Relay theo lệnh nhận về. ESP8266 giao tiếp với tầng trung gian hoàn toàn qua giao thức MQTT.

Tầng Xử lý – Backend Server (Application Layer): Node.js/Express.js Server đảm nhiệm đồng thời ba vai trò: MQTT Client (Subscribe/Publish với ESP8266 qua HiveMQ), Socket.io Server (đẩy dữ liệu realtime xuống Web UI) và RESTful HTTP Server (phục vụ truy vấn dữ liệu từ Frontend, bảo vệ endpoint điều khiển bằng JWT Middleware). Backend kết nối MySQL qua connection pool để lưu trữ và truy xuất dữ liệu.

Tầng Giao diện (Presentation Layer): Web Dashboard xây dựng bằng HTML5/CSS3/JavaScript thuần, nhận dữ liệu realtime qua Socket.io và gửi lệnh điều khiển lên Backend qua WebSocket event. Giao diện bao gồm bốn tab: Dashboard, Data Sensor, Action History và Profile.

2.2 Sơ đồ chân kết nối phần cứng

Bảng 3 Sơ đồ chân kết nối ESP8266

Thiết bị / Module	Chân Module	Chân ESP8266	Device ID	Chức năng
Cảm biến DHT11	DATA	D2 (GPIO 4)	—	Đo nhiệt độ & độ ẩm

Thiết bị / Module	Chân Module	Chân ESP8266	Device ID	Chức năng
Quang trở LDR	Tín hiệu Analog	A0 (ADC0)	—	Đo cường độ ánh sáng
LED / Relay 1	Chân dương (+)	D5 (GPIO 14)	1	Điều khiển Relay Fan
LED / Relay 2	Chân dương (+)	D6 (GPIO 12)	2	Điều khiển Relay Light

2.3 Thiết kế giao thức MQTT

Hệ thống sử dụng HiveMQ Public Broker (broker.hivemq.com:1883) với bốn topic được phân chia rõ ràng theo chức năng:

Bảng 4 Mô tả các topic MQTT

Topic	Publisher	Subscriber	Mô tả
vanh_iot/data_sensor	ESP8266	Backend	Dữ liệu cảm biến định kỳ 2 giây
vanh_iot/device_control	Backend	ESP8266	Lệnh điều khiển relay
vanh_iot/device_response	ESP8266	Backend	Xác nhận (ACK) kết quả thực thi
vanh_iot/device_sync	ESP8266	Backend	Yêu cầu đồng bộ trạng thái khi ESP8266 khởi động lại

Backend được cấu hình với clientId ngẫu nhiên, clean: true và reconnectPeriod: 1000ms để đảm bảo tự động kết nối lại khi mất kết nối với Broker.

2.4 Thiết kế RESTful API

Hệ thống cung cấp bốn nhóm endpoint HTTP phục vụ các nhu cầu khác nhau. Endpoint điều khiển thiết bị được bảo vệ bởi JWT Middleware.

Bảng 5 Mã trạng thái HTTP áp dụng

Mã	Phân loại	Diễn giải
200 OK	Thành công	Yêu cầu được thực thi trọn vẹn, dữ liệu trả về hợp lệ
400 Bad Request	Lỗi yêu cầu	Thiếu hoặc sai định dạng tham số đầu vào
401 Unauthorized	Chưa xác thực	Thiếu JWT Token trong header
403 Forbidden	Từ chối truy cập	JWT Token không hợp lệ hoặc đã hết hạn
500 Internal Error	Lỗi máy chủ	Lỗi CSDL hoặc xử lý nội bộ

2.4.1 API Xác thực (Authentication)

Mô tả: Đăng nhập bằng tài khoản admin, nhận JWT Token dùng cho các API được bảo vệ. Token có thời hạn 1 ngày, được ký với secret key vanh_secret_key_123.

Method: POST

Endpoint: /api/auth/login

Body (JSON):

```
{
  "username": "admin",
  "password": "123456"
}
```

Response (Thành công – 200 OK):

```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response (Lỗi – 401 Unauthorized):

```
{
  "success": false,
  "message": "Sai tài khoản hoặc mật khẩu"
}
```

2.4.2 API Dữ liệu cảm biến

Mô tả: Truy xuất lịch sử dữ liệu cảm biến từ CSDL, hỗ trợ phân trang và tìm kiếm theo giá trị hoặc thời gian.

Method: GET

Endpoint: /api/sensors

Bảng 6 Bảng các tham số API Dữ liệu cảm biến

Tham số	**Kiểu**	**Mặc định**	**Mô tả**
page	int	1	Số trang hiện tại
limit	int	20	Số bản ghi mỗi trang
field	string	—	Trường tìm kiếm (Time / Temperature / Humidity / Ambient Light)
keyword	string	—	Từ khóa tìm kiếm

Response (Thành công – 200 OK):

```
{
```

```
"total": 1250,
"page": 1,
"limit": 20,
"data": [
 {
  "data_id": 1024,
  "sensor_type": "Temperature",
  "unit": "°C",
  "reading_value": 28.5,
  "recorded_at": "2026-10-21 14:32:07"
 },
 {
  "data_id": 1023,
  "sensor_type": "Humidity",
  "unit": "%",
  "reading_value": 65.0,
  "recorded_at": "2026-10-21 14:32:07"
 }
]
}
```


Response (Lỗi – 500 Internal Server Error):


```
{
 "error": "Database connection failed"
}
```


2.4.3 API Lịch sử điều khiển

Mô tả: Truy xuất nhật ký các thao tác đóng/ngắt relay, hỗ trợ lọc đa tiêu chí và phân trang.

Method: GET

Endpoint: /api/actions

Bảng 7 Bảng tham số API Lịch sử điều khiển

Tham số	**Kiểu**	**Mặc định**	**Mô tả**
page	int	1	Số trang hiện tại
limit	int	20	Số bản ghi mỗi trang
device	string	All	Lọc theo tên thiết bị (Relay Fan 01 / Relay Light 01)
action	string	All	Lọc theo hành động (ON / OFF)
status	string	All	Lọc theo trạng thái (SUCCESS / LOADING / FAILED)
keyword	string	—	Tìm kiếm theo thời gian thực hiện

Response (Thành công – 200 OK):


```
{
 "total": 2847,
 "page": 1,
 "limit": 20,
 "data": [
  {
   "action_id": 501,
   "device_name": "Relay Fan 01",
   "user_name": "Trần Việt Anh",
   "action": "ON",
   "status": "SUCCESS",
   "executed_at": "2026-10-21 14:32:07"
  },
  {
   "action_id": 500,
   "device_name": "Relay Light 01",
   "user_name": "Trần Việt Anh",
   "action": "OFF",
   "status": "FAILED",
   "executed_at": "2026-10-21 14:30:15"
  }
```

```
  ]
}
```


Response (Lỗi – 500 Internal Server Error):


```
{
  "error": "Database connection failed"
}
```


2.4.4 API Thiết bị

Mô tả – GET: Truy xuất danh sách toàn bộ thiết bị và trạng thái hiện tại. Được gọi khi Web UI khởi động hoặc tải lại trang để đồng bộ vị trí công tắc relay.

Method: GET

Endpoint: /api/devices

Response (Thành công – 200 OK):


```
[
  {
    "device_id": 1,
    "device_name": "Relay Fan 01",
    "pin": 14,
    "status": 1,
    "last_connected": "2026-10-21 14:32:07"
  },
  {
    "device_id": 2,
    "device_name": "Relay Light 01",
    "pin": 12,
    "status": 0,
    "last_connected": "2026-10-21 14:30:15"
  }
]
```


Lưu ý: Trường status nhận giá trị 1 (ON) hoặc 0 (OFF).

Mô tả – POST: Gửi lệnh bật/tắt thiết bị xuống ESP8266 qua MQTT. Endpoint được bảo vệ bởi JWT Middleware — yêu cầu Bearer Token hợp lệ trong header Authorization.

Method: POST

Endpoint: /api/devices/control

Headers:

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

Body (JSON):

```
{
 "deviceId": 1,
 "action": "ON"
}
```

Response (Thành công – 200 OK):

```
{
 "success": true,
 "message": "Đã thực hiện lệnh ON cho thiết bị 1",
 "data": {
  "deviceId": 1,
  "action": "ON"
 }
}
```

Response (Lỗi – 401 Unauthorized):

```
{
 "message": "Thiếu Token!"
}
```

Response (Lỗi – 403 Forbidden):

```
{
 "message": "Token không hợp lệ!"
}
```

2.5 Luồng dữ liệu chi tiết

2.5.1 Luồng thu thập và hiển thị dữ liệu cảm biến

Định kỳ mỗi 2 giây, ESP8266 đọc các thông số từ cảm biến và Publish payload JSON lên topic vanh_iot/data_sensor. Backend (đã Subscribe topic này) tiếp nhận gói tin và thực hiện đồng thời hai tác vụ: INSERT ba bản ghi vào bảng sensor_data trong MySQL và emit sự kiện sensor_telemetry qua Socket.io tới toàn bộ Web Client đang kết nối. Web UI lắng

nghe sự kiện này và cập nhật ngay lập tức các thẻ số liệu cũng như vẽ thêm điểm mới trên biểu đồ đường mà không cần tải lại trang.

Đồng thời, sau mỗi lần nhận dữ liệu, Backend kiểm tra giá trị ánh sáng: nếu dưới 50 Lux, hệ thống emit thêm sự kiện alert_trigger xuống Web UI và tự động phát lệnh bật Relay Light nếu đèn đang tắt.

2.5.2 Luồng điều khiển thiết bị hai chiều

Khi người dùng kích hoạt công tắc toggle trên Dashboard, Web UI emit sự kiện send_control qua Socket.io lên Backend, đồng thời khóa nút và bật hiệu ứng Loading spinner. Backend xử lý theo các bước:

1. Truy vấn DB lấy tên thiết bị và trạng thái hiện tại (previousStatus) để dự phòng rollback.
2. INSERT bản ghi đầu tiên vào action_history với status = 'LOADING', emit action_logged về UI.
3. Publish lệnh JSON vào topic vanh_iot/device_control.
4. Đặt bộ đếm timeout 3.5 giây, lưu thông tin pending vào Map với key {deviceId}_{action}.
5. ESP8266 nhận lệnh, kích hoạt relay và Publish ACK vào topic vanh_iot/device_response.
6. Backend nhận ACK, xóa pending khỏi Map, UPDATE trạng thái trong bảng device, INSERT bản ghi SUCCESS vào action_history và emit device_state_sync về UI để mở khóa giao diện.
7. Nếu hết timeout mà không nhận được ACK, Backend INSERT bản ghi FAILED, revert device.status về previousStatus và emit device_state_sync với reason: 'TIMEOUT_FAILED' để UI khôi phục công tắc về vị trí cũ và hiển thị Toast thông báo lỗi.

2.5.3 Luồng đồng bộ trạng thái khi ESP8266 khởi động

Khi ESP8266 kết nối lại sau khi mất điện hoặc restart, thiết bị Publish một gói tin vào topic vanh_iot/device_sync. Backend nhận tín hiệu này, truy vấn toàn bộ trạng thái thiết bị trong DB và gửi lần lượt từng lệnh tương ứng xuống topic device_control, đảm bảo phần cứng luôn đồng bộ với trạng thái logic trên server mà không cần can thiệp thủ công.

CHƯƠNG 3. THIẾT KẾ CHI TIẾT

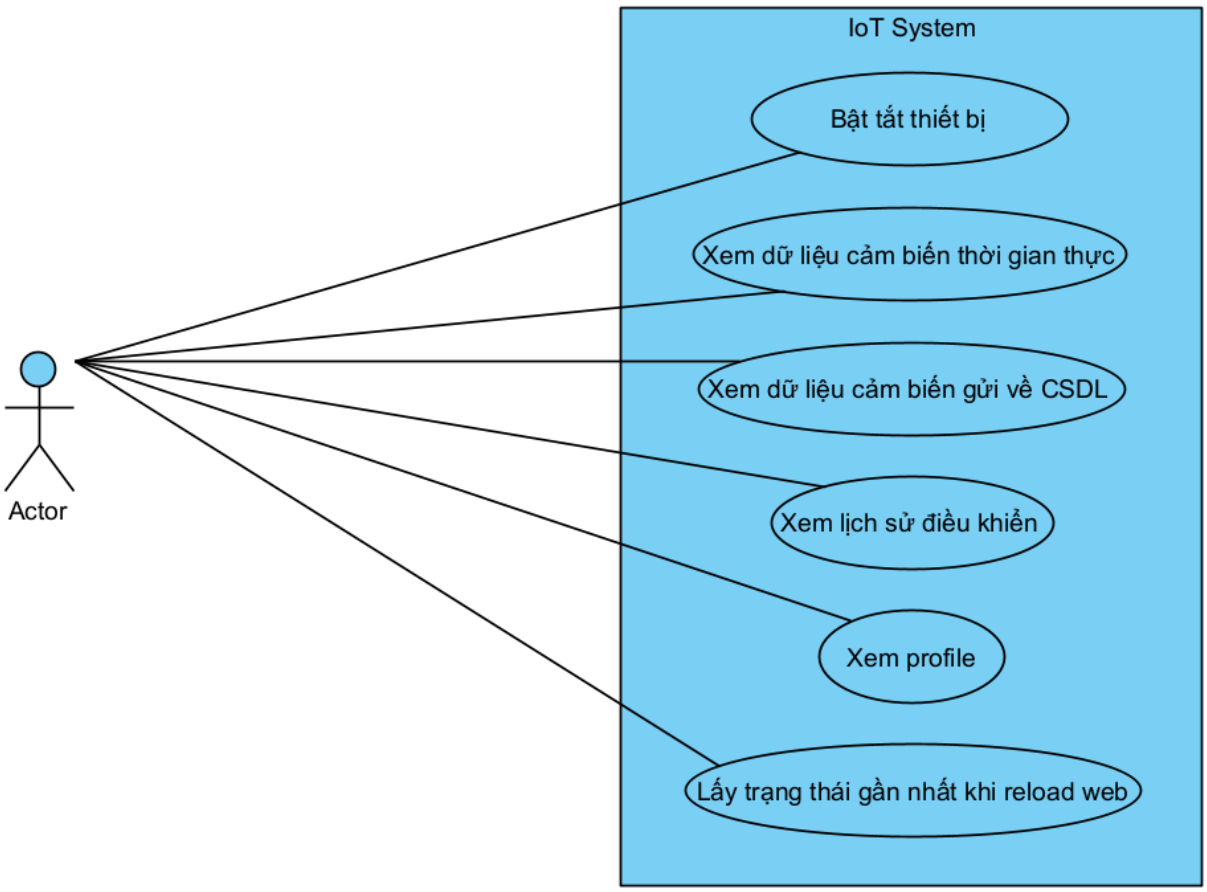
3.1 Phân tích yêu cầu và Use Case

3.1.1 Các tác nhân trong hệ thống

Bảng 8 Mô tả các tác nhân

Tác nhân	**Loại**	**Mô tả**
User (Người dùng)	Tác nhân chính	Truy cập Web Dashboard để giám sát và điều khiển thiết bị
ESP8266	Tác nhân phần cứng	Vi điều khiển thu thập dữ liệu cảm biến và thực thi lệnh relay
MQTT Broker (HiveMQ)	Tác nhân hệ thống	Trung gian truyền thông điệp giữa ESP8266 và Backend
Backend Server	Tác nhân hệ thống	Xử lý logic nghiệp vụ, lưu trữ CSDL, trung chuyển dữ liệu

3.1.2 Sơ đồ Use Case tổng quan



Hình 2 Sơ đồ Use Case tổng quan

3.1.3 Đặc tả chi tiết Use Case

Bảng 9 Đặc tả Use Case: Điều khiển thiết bị

Tên usecase	Điều khiển thiết bị
Actor	User
Mô tả	Cho phép người dùng điều khiển đóng/ngắt các thiết bị ngoại vi từ xa thông qua giao diện Web. Hệ thống ghi nhận mọi biến động trạng thái dưới dạng các bản ghi nối tiếp (Append-Only Log).
Tiền điều kiện	Người dùng đang truy cập Dashboard. Thiết bị ESP32 được cấp nguồn và duy trì kết nối mạng với MQTT Broker.
Hậu điều kiện	Thiết bị phần cứng thay đổi trạng thái, 2 bản ghi lịch sử nối tiếp (LOADING và SUCCESS/FAILED) được thêm mới vào cơ sở dữ liệu.
Kịch bản chính	1. User click vào công tắc (ON/OFF) của thiết bị trên WebUI. 2. Giao diện chuyển nút sang trạng thái LOADING (khóa nút bấm) và gửi POST request lên Backend. 3. Backend thực hiện INSERT bản ghi thứ nhất vào bảng ActionHistory với trạng thái LOADING. 4. Backend publish lệnh điều khiển (JSON) vào topic device_control qua MQTT Broker. 5. ESP32 nhận lệnh và kích hoạt Relay phần cứng. 6. ESP32 publish phản hồi kết quả vào topic device_response. 7. Backend nhận xác nhận, thực hiện INSERT bản ghi thứ hai vào bảng ActionHistory với trạng thái SUCCESS/FAILED. 8. Backend trả HTTP 200 về WebUI để mở khóa nút và gạt công tắc sang vị trí mới.
Ngoại lệ	- Quá hạn (Timeout): Quá 5s không có phản hồi từ ESP32, Backend lưu trạng thái TIMEOUT, UI khôi phục nút về cũ và báo lỗi. - Lỗi phần cứng: ESP32 trả về lỗi, Backend lưu trạng thái FAILED, UI báo lỗi.

Bảng 10 Đặc tả Use Case: Xem dữ liệu cảm biến thời gian thực

Tên usecase	Xem dữ liệu cảm biến thời gian thực
Actor	User
Mô tả	Người dùng theo dõi sự thay đổi của các chỉ số môi trường (Nhiệt độ, Độ ẩm, Ánh sáng) liên tục trên Dashboard.

Tiền điều kiện	Người dùng đang truy cập Dashboard, các cảm biến hoạt động bình thường, kết nối MQTT và WebSocket ổn định.
Hậu điều kiện	Dữ liệu mới nhất được lưu vào CSDL và cập nhật tức thời lên biểu đồ, thẻ thông tin trên UI.
Kịch bản chính	1. ESP32 đọc dữ liệu từ cảm biến định kỳ mỗi 2 giây. 2. ESP32 đóng gói payload JSON và Publish lên topic data_sensor. 3. Backend (đã Subscribe) tiếp nhận gói tin, bóc tách và INSERT vào bảng SensorData. 4. Cùng lúc, Backend đẩy dữ liệu qua WebSocket sang WebUI. 5. WebUI nhận luồng dữ liệu, cập nhật thẻ số và vẽ tiếp điểm mới trên Line Chart không cần tải lại trang.
Ngoại lệ	- Mất kết nối mạng, luồng WebSocket ngắt, dữ liệu trên màn hình sẽ đứng yên ở mốc cuối cùng nhận được.

Bảng 11 Đặc tả Use Case: Xem dữ liệu cảm biến lưu trong CSDL

Tên usecase	Xem dữ liệu cảm biến gửi về CSDL
Actor	User
Mô tả	Truy xuất, rà soát lại toàn bộ lịch sử các mốc đo lường cảm biến đã được lưu trữ trong quá khứ.
Tiền điều kiện	Truy cập thành công vào tab "Data Sensor" trên thanh điều hướng.
Hậu điều kiện	Danh sách dữ liệu hiển thị thành bảng chi tiết, có phân trang.
Kịch bản chính	6. User click điều hướng sang trang Data Sensor. 7. WebUI gửi API request lấy dữ liệu (bao gồm các biến phân trang page, limit). 8. Backend kết nối CSDL, truy vấn bảng SensorData. 9. Backend trả về khối dữ liệu JSON cho WebUI. 10. WebUI render dữ liệu ra bảng, hiển thị theo từng loại cảm biến và thời gian.
Ngoại lệ	- Không có dữ liệu trong Database, hệ thống trả về mảng rỗng và hiển thị bảng trống (No data).

Bảng 12 Đặc tả Use Case: Xem lịch sử điều khiển

Tên usecase	Xem lịch sử điều khiển
Actor	User
Mô tả	Người dùng kiểm tra lại nhật ký các phiên thao tác điều khiển thiết bị (ai thao tác, thiết bị nào, thời gian, kết quả).
Tiền điều kiện	Truy cập thành công vào tab "Action History" trên thanh điều hướng.
Hậu điều kiện	Bảng lịch sử điều khiển được tải lên giao diện với định dạng màu sắc phân biệt theo trạng thái (SUCCESS, FAILED, TIMEOUT).
Kịch bản chính	11. User truy cập vào trang Action History. 12. WebUI gọi API request lấy nhật ký điều khiển, kèm tham số lọc (nếu có). 13. Backend truy vấn bảng ActionHistory (kết hợp bảng Device để lấy tên). 14. Trả mảng dữ liệu JSON về cho Frontend. 15. Frontend render bảng, hiển thị đầy đủ thông tin Log ID, tên thiết bị, hành động và trạng thái thi hành.
Ngoại lệ	- Lỗi kết nối CSDL, trả về HTTP 500, WebUI hiển thị popup thông báo gián đoạn hệ thống.

Bảng 13 Đặc tả Use Case: Đồng bộ trạng thái khi tải lại trang

Tên usecase	Lấy trạng thái gần nhất khi reload web
Actor	User
Mô tả	Tự động đồng bộ toàn bộ giao diện (trạng thái công tắc ON/OFF, chỉ số cảm biến) khớp với thực tế thiết bị ngay khi user F5 tải lại trang.
Tiền điều kiện	User tải lại trang trình duyệt (Reload/F5).
Hậu điều kiện	Giao diện hiển thị chính xác thiết bị nào đang bật/tắt ở thời điểm hiện tại.
Kịch bản chính	16. Trình duyệt tải lại DOM và vòng đời của WebUI khởi động. 17. Quá trình mounted gọi ngay API GET /telemetry/latest. 18. Backend truy xuất bản ghi cảm biến mới nhất và trạng thái cuối cùng của thiết bị trong DB. 19. Trả dữ liệu khởi tạo về. 20. WebUI gán giá trị này để hiển thị màu sắc/vị trí của công tắc Relay.

Ngoại lệ	- API lỗi hoặc mạng rớt ngay lúc tải, WebUI gán giá trị mặc định (N/A, OFF) và chờ tín hiệu WebSocket tiếp theo

3.2 Thiết kế cơ sở dữ liệu

3.2.1 Mô tả các bảng

Bảng 14 Mô tả bảng user

Cột	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả**
user_id	INT(10)	PK, AUTO_INCREMENT	Định danh người dùng
user_name	VARCHAR(50)	NOT NULL	Tên hiển thị
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã hash
email	VARCHAR(100)	NOT NULL	Địa chỉ email
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm tạo tài khoản

Bảng 15 Mô tả bảng device

Cột	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả**
device_id	INT(10)	PK, AUTO_INCREMENT	Định danh thiết bị
device_name	VARCHAR(50)	NULL	Tên thiết bị
pin	INT(10)	NOT NULL	Chân GPIO điều khiển
status	TINYINT(1)	DEFAULT 0	Trạng thái hiện tại (0=OFF, 1=ON)
last_connected	TIMESTAMP	ON UPDATE NOW()	Lần cập nhật trạng thái gần nhất

Bảng 16 Mô tả bảng sensor

Cột	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả**
sensor_id	INT(10)	PK, AUTO_INCREMENT	Định danh cảm biến
sensor_type	VARCHAR(50)	NOT NULL	Loại cảm biến (Temperature / Humidity / Ambient Light)

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
unit	VARCHAR(50)	NULL	Đơn vị đo lường

Bảng 17 Mô tả bảng sensor_data

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
data_id	BIGINT(19)	PK, AUTO_INCREMENT	Định danh bản ghi đo lường
sensor_id	INT(10)	FK → sensor	Tham chiếu loại cảm biến
reading_value	FLOAT	NOT NULL	Giá trị đo được
recorded_at	TIMESTAMP	DEFAULT NOW()	Thời điểm ghi nhận (múi giờ VN)

Bảng 18 Mô tả bảng action_history

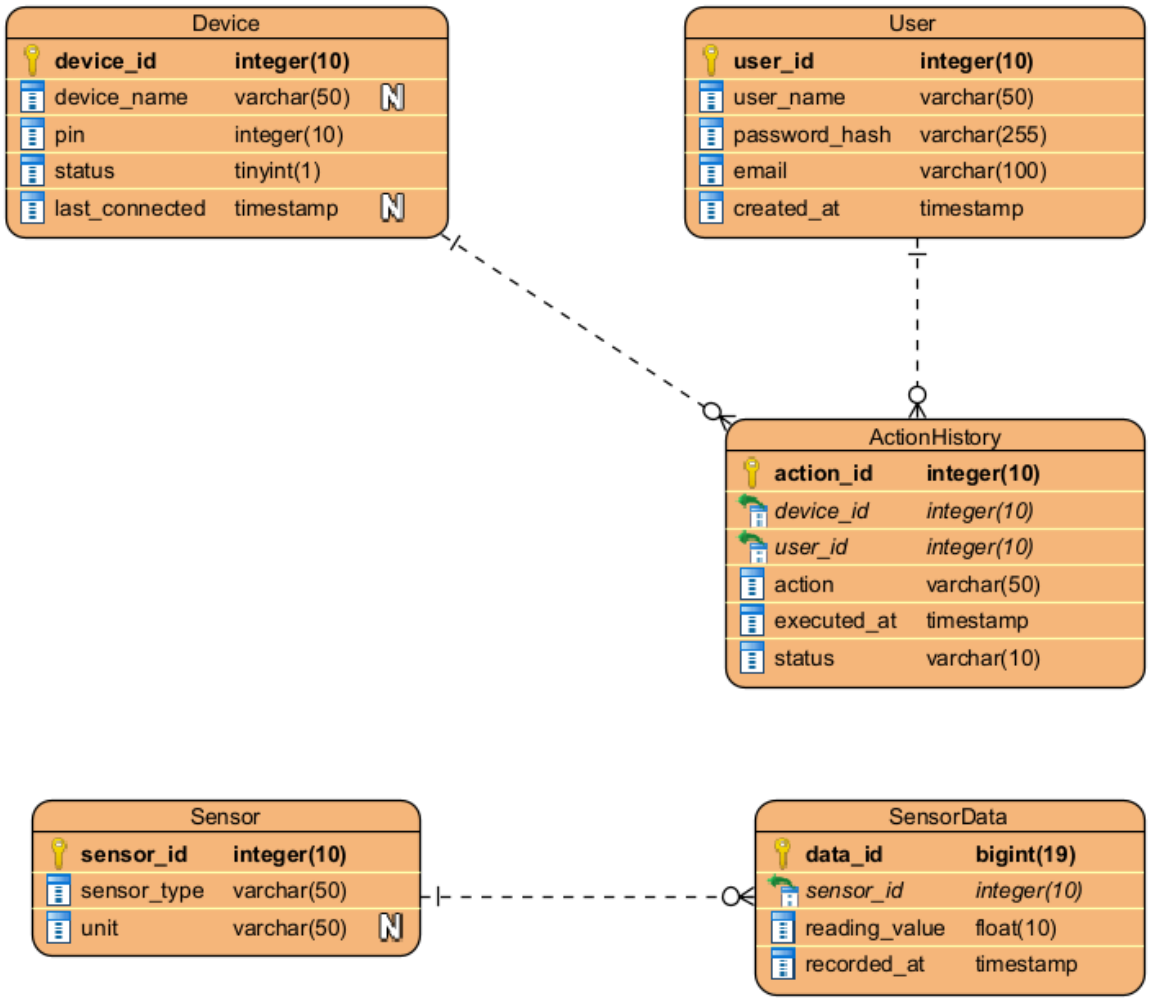
Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
action_id	INT(10)	PK, AUTO_INCREMENT	Định danh bản ghi lịch sử
device_id	INT(10)	FK → device	Thiết bị được điều khiển
user_id	INT(10)	FK → user	Người thực hiện thao tác
action	VARCHAR(50)	NOT NULL	Hành động (ON / OFF)
executed_at	TIMESTAMP	DEFAULT NOW()	Thời điểm thực hiện
status	VARCHAR(10)	NOT NULL	Kết quả (SUCCESS / LOADING / FAILED)

3.2.2 Quan hệ giữa các bảng

User – ActionHistory (1:N): Một người dùng có thể thực hiện nhiều thao tác điều khiển. Mỗi bản ghi trong action_history tham chiếu người dùng qua khóa ngoại user_id.

Device – ActionHistory (1:N): Một thiết bị có thể tiếp nhận nhiều lượt điều khiển. Mỗi bản ghi lịch sử tham chiếu thiết bị qua khóa ngoại device_id.

Sensor – SensorData (1:N): Mỗi cảm biến tạo ra nhiều bản ghi đo lường theo chu kỳ. Các bản ghi tham chiếu cảm biến qua khóa ngoại sensor_id.



Hình 3 Sơ đồ ERD

3.3 Thiết kế và cài đặt Backend

3.3.1 Cấu trúc thư mục dự án

IoT/

```
├── config/
│   ├── db.js
│   └── mqtt.js
├── controllers/
│   ├── sensorController.js
│   ├── actionController.js
│   └── deviceController.js
├── middlewares/
│   └── authMiddleware.js
├── mqtt/
```

```
│   └── mqttHandler.js
├── routes/
│   ├── sensorRoutes.js
│   ├── actionRoutes.js
│   └── deviceRoutes.js
├── services/
│   └── deviceService.js
├── sockets/
│   └── socketHandler.js
├── utils/
│   └── timeUtil.js
├── public/
├── swagger.json
├── server.js
└── package.json
```

Kiến trúc mã nguồn tuân theo mô hình phân tầng: Routes chỉ định tuyến request → Middlewares kiểm tra xác thực → Controllers xử lý logic nghiệp vụ → Services thực hiện tác vụ phức tạp dùng chung.

3.3.2 Một số điểm kỹ thuật đáng chú ý

JWT Authentication Middleware: File middlewares/authMiddleware.js kiểm tra header Authorization: Bearer <token>, verify bằng secret key vanh_secret_key_123 và đính kèm thông tin user vào req.user cho controller sử dụng. Endpoint POST /api/devices/control được bảo vệ bởi middleware này.

Pending Actions Map: deviceService.js sử dụng Map trong bộ nhớ để theo dõi các lệnh đang chờ phản hồi với key {deviceId}_{action} (ví dụ: 1_ON). Cơ chế này ngăn gửi lệnh trùng lặp khi người dùng bấm nhanh, đồng thời lưu previousStatus để rollback khi timeout.

Xử lý múi giờ Việt Nam: Toàn bộ timestamp được chuẩn hóa về Asia/Ho_Chi_Minh qua getVNTimeString() trước khi INSERT vào MySQL, đảm bảo nhất quán giữa dữ liệu lưu trữ và thời gian hiển thị trên giao diện.

Mock Device Script: File mock.js mô phỏng hành vi ESP8266 trong môi trường không có phần cứng thực — Subscribe topic device_control, nhận lệnh và sau 300ms tự động Publish ACK SUCCESS về topic device_response. Công cụ này hữu ích cho việc kiểm thử flow điều khiển mà không cần kết nối phần cứng vật lý.

3.3.3 Danh sách thư viện phụ thuộc

Bảng 19 Danh sách thư viện phụ thuộc

Thư viện	Phiên bản	Mục đích
express	5.2.1	Web framework xây dựng REST API
socket.io	4.8.3	WebSocket server cho realtime communication
mqtt	5.15.2	MQTT client kết nối HiveMQ Broker
mysql2	3.24.2	MySQL driver hỗ trợ Promise/async-await
jsonwebtoken	—	Ký và verify JWT Token
swagger-ui-express	5.0.1	Render Swagger UI từ swagger.json
cors	2.8.6	Xử lý Cross-Origin Resource Sharing

3.4 Thiết kế giao diện người dùng

Giao diện Web Dashboard sử dụng Dark Theme hiện đại với hệ thống màu sắc nhất quán: nền xanh đen đậm (#0B1120), sidebar (#0F172A), card (#1E293B) kết hợp các màu nhấn xanh dương (#38BDF8), xanh lá (#10B981), đỏ (#EF4444) và vàng (#F59E0B) để phân biệt trạng thái và loại dữ liệu. Font chữ Inter được nhúng từ Google Fonts.

Tab Dashboard (Telemetry Console): Ba thẻ số liệu cảm biến cập nhật realtime qua WebSocket, biểu đồ đường Chart.js với hai trục Y độc lập (trục trái cho nhiệt độ/độ ẩm 0–100, trục phải cho ánh sáng 0–1000 Lux), và hai công tắc toggle điều khiển relay với hiệu ứng Loading spinner.

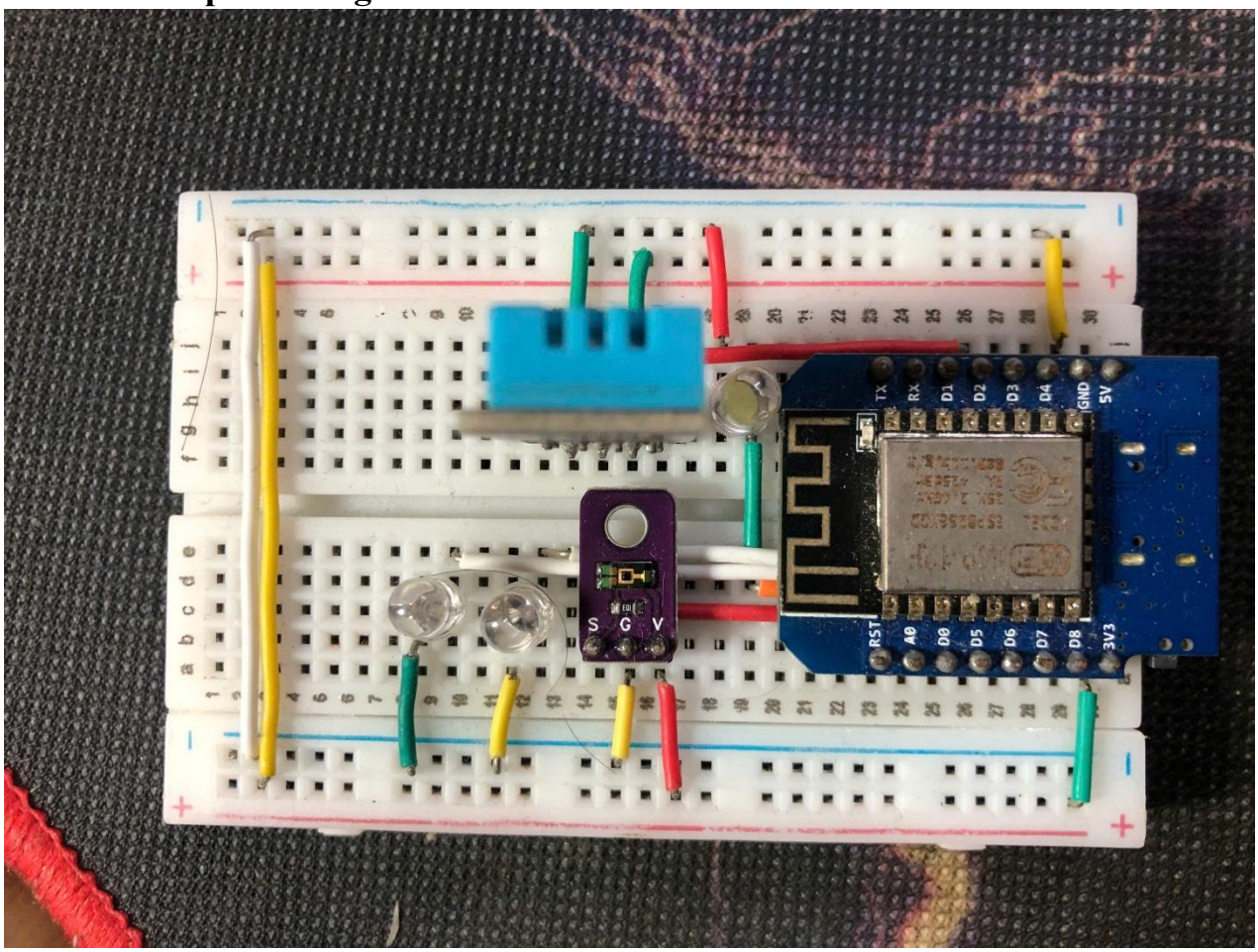
Tab Data Sensor: Bảng lịch sử cảm biến với tìm kiếm, lọc theo loại và phân trang tùy chỉnh (10/20/30/tùy chỉnh tối đa 50 dòng/trang).

Tab Action History: Bảng nhật ký điều khiển với bộ lọc đa tiêu chí đồng thời (thiết bị, hành động, trạng thái, thời gian) và badge màu phân biệt trạng thái.

Tab Profile: Thẻ thông tin sinh viên, chi tiết học thuật, liên kết dự án (GitHub, Figma, Swagger, PDF báo cáo) và tech stack.

CHƯƠNG 4. KẾT QUẢ

4.1 Mô hình phần cứng



Hình 4 Mô hình phần cứng

4.2 Kết quả triển khai hệ thống

Sau khi hoàn thiện phát triển, hệ thống được triển khai và kiểm thử trên môi trường local với đầy đủ các thành phần. Hệ thống khởi động thành công với thông báo:

[HTTP Server] Running at: http://localhost:5000

[MQTT Info] Backend da KET NOI THANH CONG toi HiveMQ Broker!

4.3 Giao diện Dashboard – Telemetry Console

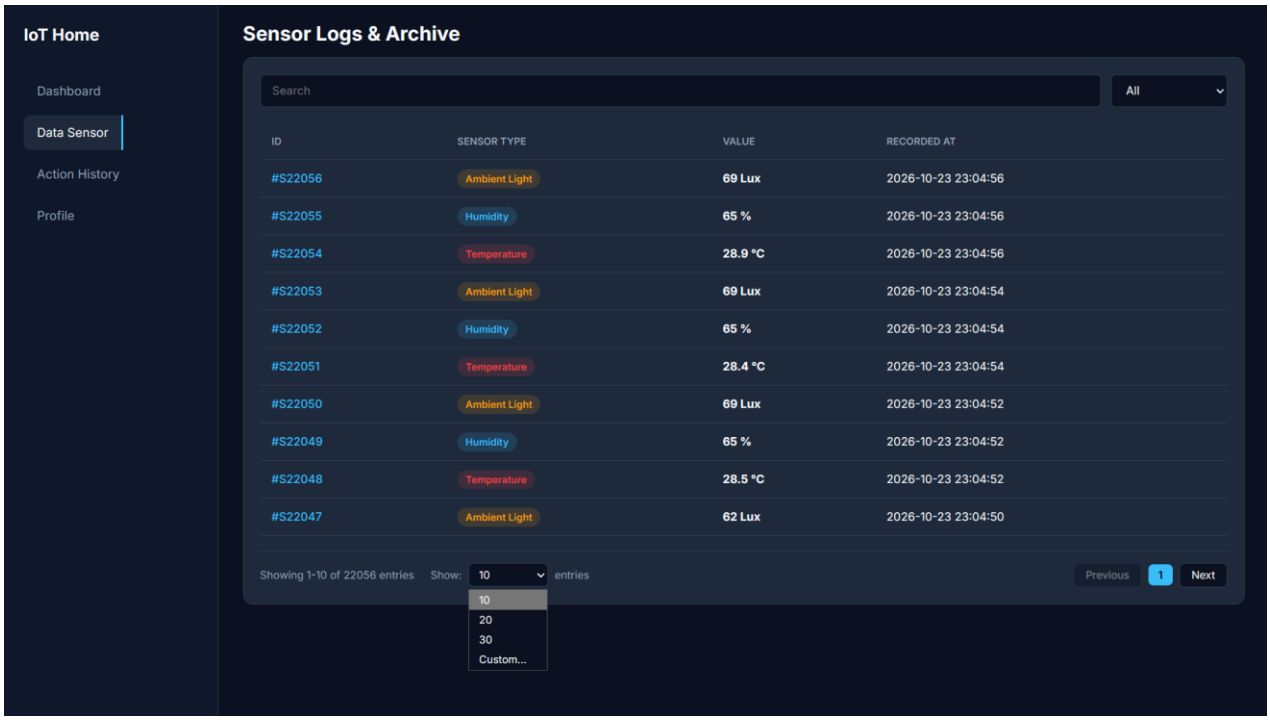
Tab Dashboard là giao diện chính, hiển thị ba thẻ số liệu cảm biến cập nhật tự động mỗi 2 giây qua WebSocket. Khi cường độ ánh sáng xuống dưới 50 Lux, thẻ Ambient Light chuyển sang viền đỏ với hiệu ứng nhấp nháy pulseAlert. Biểu đồ đường giữ tối đa 8 điểm dữ liệu gần nhất và trượt theo thời gian thực.



Hình 5 Giao diện Dashboard

4.4 Giao diện Tab Data Sensor

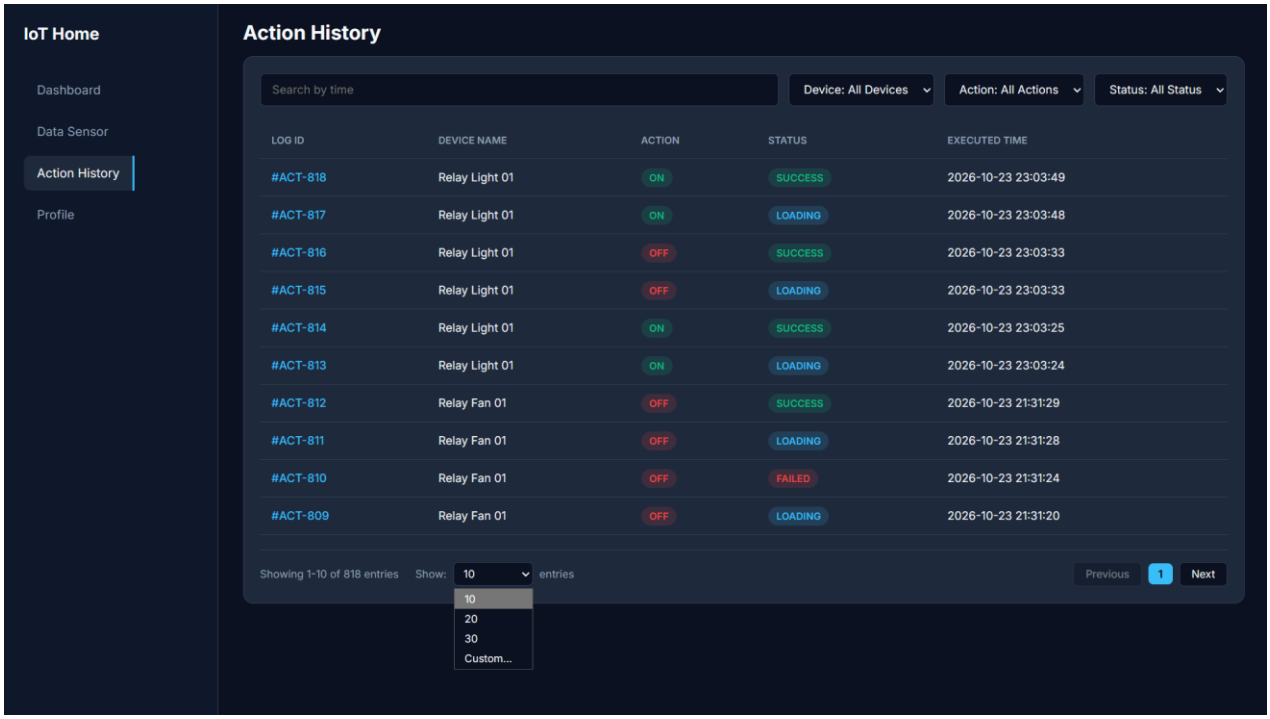
Bảng dữ liệu lịch sử cảm biến với chức năng tìm kiếm và lọc theo loại. Mỗi loại cảm biến được phân biệt bằng màu badge: đỏ cho Temperature, xanh dương cho Humidity, vàng cho Ambient Light. Thông tin phân trang hiển thị đầy đủ "Showing X–Y of Z entries".



Hình 6 Giao diện Tab Data Sensor

4.5 Giao diện Tab Action History

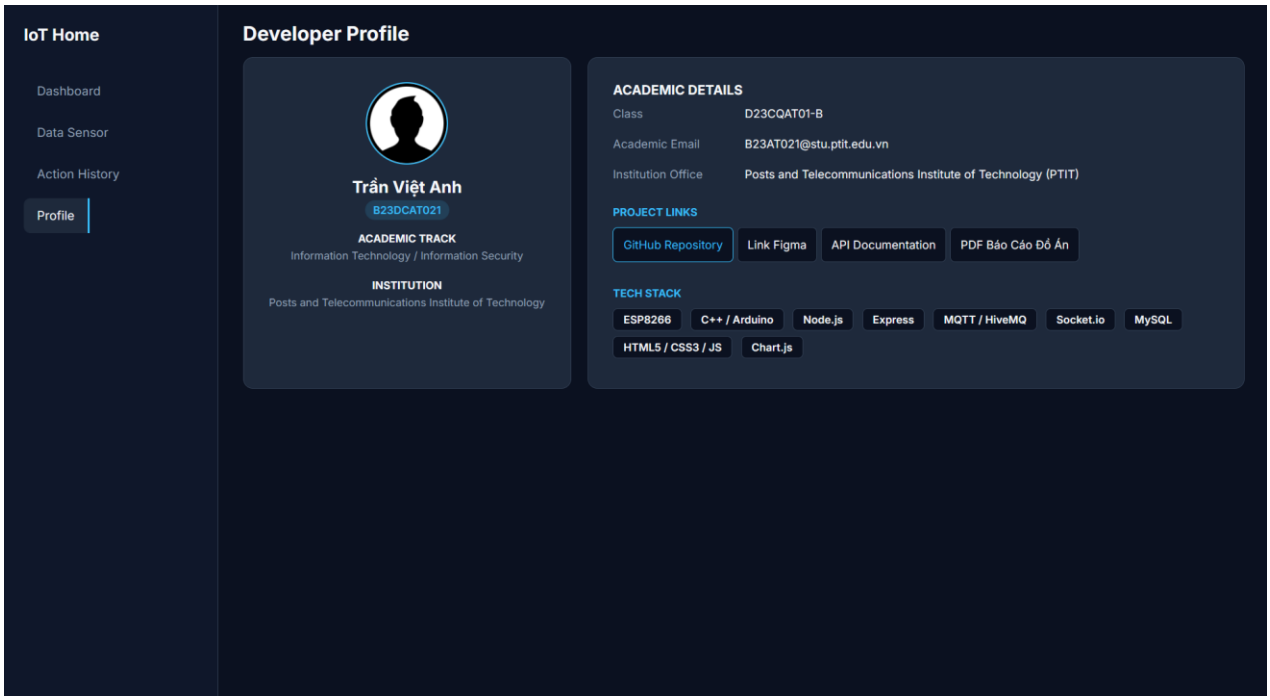
Bảng nhật ký điều khiển với bốn bộ lọc đồng thời: tên thiết bị, loại hành động, trạng thái và tìm kiếm theo thời gian. Trạng thái SUCCESS hiển thị badge xanh lá, FAILED hiển thị đỏ, LOADING hiển thị xanh dương.



Hình 7 Giao diện Tab Action History

4.6 Giao diện Tab Developer Profile

Tab Profile hiển thị thẻ thông tin định danh nhà phát triển với ảnh đại diện, mã sinh viên, thông tin học thuật, các liên kết dự án và tech stack.



Hình 8 Giao diện Tab Developer Profile

4.7 Cơ chế Loading và xác nhận trạng thái

Khi người dùng kích hoạt công tắc relay, giao diện phản hồi tức thì bằng hiệu ứng spinner xoay trên thanh trượt toggle và khóa nút bấm. Sau khi nhận ACK SUCCESS từ ESP8266 (thường trong 0.5–1 giây), spinner biến mất và trạng thái cập nhật. Nếu timeout 3.5 giây, công tắc tự revert và Toast Notification đỏ xuất hiện ở góc trên phải.



Hình 9 Trạng thái Loading khi điều khiển thiết bị

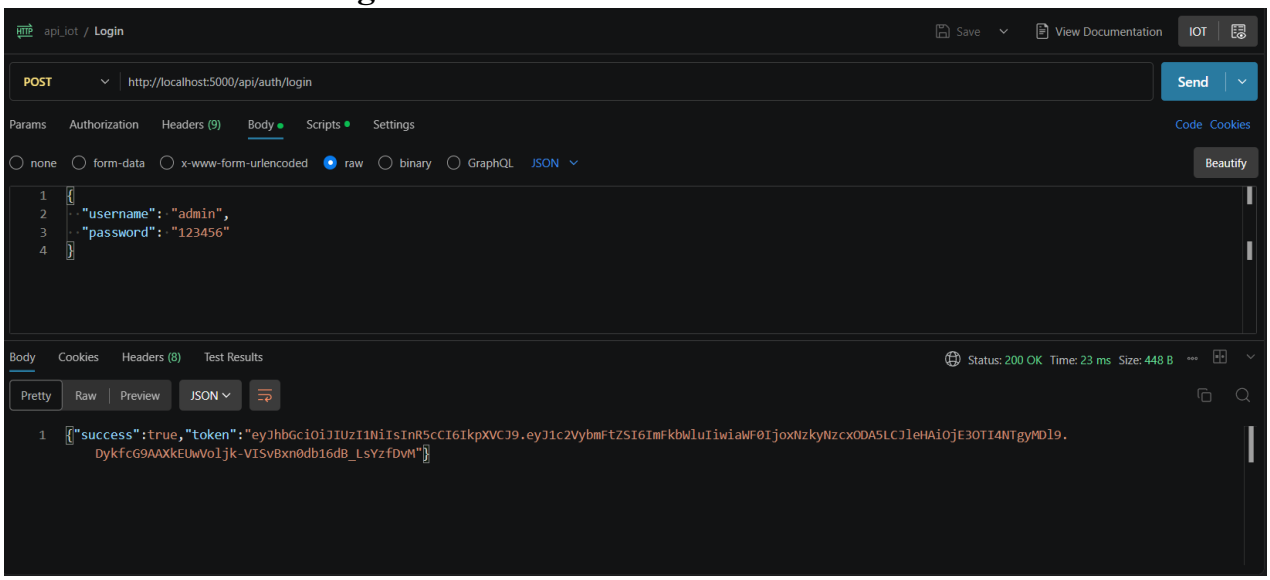
4.8 Cảnh báo tự động ánh sáng thấp

Khi cường độ ánh sáng dưới 50 Lux, hệ thống đồng thời: hiển thị Toast Notification màu vàng cảnh báo, đánh dấu thẻ Ambient Light với viền đỏ nhấp nháy và tự động phát lệnh bật Relay Light 01 nếu đèn đang tắt và không có lệnh nào đang pending.



Hình 10 Toast Notification cảnh báo ánh sáng thấp

4.9 Kiểm thử API bằng Postman



Hình 11 Kiểm thử API – Đăng nhập lấy JWT Token



Hình 12 Kiểm thử API – Điều khiển thiết bị với Bearer Token

Bước 1 – Đăng nhập:

POST http://localhost:5000/api/auth/login

Body: {"username": "admin", "password": "123456"}

→ Response: {"success": true, "token": "eyJhbGci..."}

Bước 2 – Điều khiển thiết bị:

POST http://localhost:5000/api/devices/control

Authorization: Bearer eyJhbGci...

Body: {"deviceId": 1, "action": "ON"}

→ Response: {"success": true, "message": "Đã thực hiện lệnh ON cho thiết bị 1", ...}

Nếu gọi endpoint điều khiển mà không có Token, hệ thống trả về 401 Unauthorized: {"message": "Thiếu Token!"}. Nếu Token không hợp lệ hoặc hết hạn, trả về 403 Forbidden: {"message": "Token không hợp lệ!"}.

4.10 Đánh giá kết quả

Mục tiêu	**Kết quả**
Giám sát môi trường thời gian thực (≤ 500ms)	Dữ liệu cập nhật liên tục qua WebSocket
Điều khiển thiết bị hai chiều có xác nhận	Cơ chế LOADING → SUCCESS/FAILED hoạt động ổn định
Xác thực JWT cho API điều khiển	Endpoint POST /control được bảo vệ
Timeout và revert tự động (3.5 giây)	Giao diện khôi phục về vị trí cũ khi timeout

Mục tiêu	**Kết quả**
Tự động hóa bật đèn khi ánh sáng < 50 Lux	Kết hợp alert và auto-control
Tra cứu dữ liệu với phân trang và lọc	Hỗ trợ đầy đủ tìm kiếm đa tiêu chí
Tài liệu API chuẩn OpenAPI 3.0	Swagger UI khả dụng tại /api-docs
Đồng bộ trạng thái khi ESP8266 khởi động lại	Topic device_sync xử lý tự động

CHƯƠNG 5. LỜI KẾT

5.1 Tổng kết

Đề tài đã nghiên cứu, thiết kế và triển khai thành công hệ thống IoT phục vụ giám sát các thông số môi trường và điều khiển thiết bị thông minh trong không gian trong nhà. Hệ thống được xây dựng với kiến trúc xuyên suốt từ tầng phần cứng sử dụng vi điều khiển ESP8266 đến tầng ứng dụng Web Dashboard, bảo đảm sự phối hợp giữa các thành phần và đáp ứng các mục tiêu đề ra ban đầu.

Một trong những kết quả nổi bật của đề tài là việc kết hợp ba giao thức truyền thông MQTT, WebSocket và RESTful HTTP nhằm đáp ứng các nhu cầu trao đổi dữ liệu khác nhau. Trong đó, MQTT đảm nhiệm việc truyền nhận dữ liệu giữa thiết bị IoT và máy chủ; WebSocket hỗ trợ cập nhật trạng thái trên giao diện theo thời gian thực; RESTful HTTP cung cấp cơ chế truy vấn và trao đổi dữ liệu theo cấu trúc. Bên cạnh đó, hệ thống tích hợp cơ chế xác thực bằng JSON Web Token (JWT) để bảo vệ các chức năng điều khiển. Cơ chế xác nhận phản hồi hai chiều, kết hợp với xử lý timeout và rollback khi thao tác thất bại, góp phần hạn chế sự sai lệch giữa trạng thái hiển thị trên phần mềm và trạng thái thực tế của thiết bị.

Về mặt kiến thức và kỹ năng, đề tài là cơ hội vận dụng tổng hợp nhiều lĩnh vực như lập trình nhúng trên ESP8266, phát triển backend theo kiến trúc phân tầng, thiết kế cơ sở dữ liệu quan hệ, xây dựng API, triển khai cơ chế xác thực và phát triển giao diện web tương tác. Qua đó, đề tài đã xây dựng được nền tảng thực tiễn cho việc nghiên cứu và phát triển các hệ thống nhà thông minh trong tương lai.

5.2 Hạn chế hiện tại

Mặc dù đạt được các mục tiêu cơ bản, hệ thống vẫn tồn tại một số hạn chế cần được khắc phục để nâng cao tính bảo mật, khả năng mở rộng và mức độ ổn định khi triển khai trong môi trường thực tế.

- Về bảo mật cấu hình: Một số thông tin nhạy cảm như khóa bí mật JWT và thông tin kết nối cơ sở dữ liệu vẫn được khai báo trực tiếp trong mã nguồn. Tài khoản quản trị hiện sử dụng thông tin đăng nhập cố định (admin/123456), chưa được quản lý tập trung trong cơ sở dữ liệu và chưa áp dụng cơ chế băm mật khẩu.
- Về bảo mật truyền thông: Hệ thống sử dụng public broker của HiveMQ thông qua giao thức MQTT không mã hóa. Cách triển khai này phù hợp với mục đích thử nghiệm và học tập, tuy nhiên chưa bảo đảm yêu cầu về bảo mật dữ liệu khi triển khai trong môi trường thực tế.
- Về quản lý người dùng: Hệ thống chưa hỗ trợ đầy đủ nhiều người dùng và phân quyền truy cập. Giá trị currentUserId hiện được cố định bằng 1 trong các module deviceService.js và mqttHandler.js, do đó chưa đáp ứng được nhu cầu quản lý thiết bị và dữ liệu riêng biệt theo từng tài khoản.
- Về quản lý dữ liệu: Bảng sensor_data liên tục ghi nhận dữ liệu cảm biến với tần suất cao, ước tính khoảng 129.600 bản ghi mỗi ngày với giả định có ba bản ghi được tạo

sau mỗi chu kỳ hai giây. Tuy nhiên, hệ thống chưa triển khai chính sách lưu trữ và vòng đời dữ liệu (Data Retention), có thể dẫn đến sự gia tăng đáng kể về dung lượng cơ sở dữ liệu và ảnh hưởng đến hiệu năng truy vấn trong dài hạn.

5.3 Hướng phát triển

- Trong tương lai, hệ thống có thể tiếp tục được mở rộng theo hướng nâng cao khả năng giám sát môi trường, tăng cường tính tự động trong điều khiển thiết bị và ứng dụng các công nghệ phân tích dữ liệu hiện đại. Các định hướng phát triển cụ thể bao gồm:

5.3.1 Mở rộng khả năng giám sát môi trường

Hệ thống có thể tích hợp thêm các cảm biến như CO_2, bụi mịn PM2.5 và cảm biến chuyển động PIR nhằm cung cấp thông tin toàn diện hơn về chất lượng không khí và tình trạng hoạt động trong không gian. Việc mở rộng nguồn dữ liệu giúp hệ thống đánh giá chính xác hơn các điều kiện môi trường, đồng thời tạo cơ sở để xây dựng các chức năng điều khiển tự động dựa trên nhiều yếu tố đầu vào.

5.3.2 Phát triển cơ chế điều khiển tự động

Bên cạnh khả năng điều khiển thiết bị thông qua giao diện Web Dashboard, hệ thống có thể được bổ sung chức năng lập lịch (Scheduler), cho phép người dùng thiết lập thời gian bật, tắt thiết bị theo nhu cầu sử dụng. Đồng thời, có thể xây dựng cơ chế điều khiển dựa trên các ngưỡng môi trường được cấu hình trước, chẳng hạn tự động kích hoạt quạt khi nhiệt độ vượt ngưỡng hoặc cảnh báo khi chất lượng không khí suy giảm. Những cải tiến này sẽ giúp giảm sự phụ thuộc vào thao tác thủ công và nâng cao tính tiện nghi trong quá trình sử dụng.

5.3.3 Xây dựng hệ thống cảnh báo thông minh

Một hướng phát triển khác là xây dựng module cảnh báo cho phép người dùng thiết lập các ngưỡng cảnh báo riêng cho từng thông số môi trường. Khi dữ liệu vượt khỏi phạm vi cho phép hoặc thiết bị phát sinh trạng thái bất thường, hệ thống có thể gửi thông báo qua email hoặc Telegram. Ngoài ra, việc kết hợp dữ liệu cảm biến với lịch sử hoạt động của thiết bị sẽ hỗ trợ phát hiện các tình huống bất thường và cung cấp thông tin kịp thời cho người dùng.

5.3.4 Tối ưu khả năng lưu trữ và khai thác dữ liệu

Khi số lượng thiết bị và khối lượng dữ liệu tăng lên, hệ thống cần được phát triển theo hướng quản lý và khai thác dữ liệu hiệu quả hơn. Các giải pháp có thể bao gồm tối ưu cấu trúc cơ sở dữ liệu, bổ sung chỉ mục cho những trường thường xuyên được truy vấn và xây dựng cơ chế lưu trữ dữ liệu lịch sử phù hợp. Bên cạnh đó, dữ liệu cảm biến có thể được tổng hợp theo các khoảng thời gian để hỗ trợ trực quan hóa xu hướng nhiệt độ, độ ẩm và các thông số môi trường khác, giúp người dùng theo dõi sự thay đổi theo thời gian thuận tiện hơn.

5.3.5 Ứng dụng Machine Learning trong phân tích và dự đoán

Trên cơ sở dữ liệu lịch sử được thu thập, hệ thống có thể tích hợp các mô hình Machine Learning nhằm dự đoán xu hướng thay đổi của các thông số môi trường. Kết quả dự đoán có thể được sử dụng để đưa ra khuyến nghị vận hành thiết bị hoặc xây dựng các chiến lược

điều khiển chủ động. Ví dụ, hệ thống có thể dự đoán thời điểm nhiệt độ có khả năng vượt ngưỡng để kích hoạt thiết bị làm mát sớm hơn. Đây là định hướng giúp chuyển đổi hệ thống từ mô hình giám sát và điều khiển theo điều kiện hiện tại sang mô hình có khả năng phân tích và hỗ trợ ra quyết định.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, *ESP8266 Technical Reference*, Version 1.7, 2020. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[2] OASIS Standard, *MQTT Version 5.0*, OASIS Open, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[3] HiveMQ, *HiveMQ MQTT Broker Documentation*, 2024. [Online]. Available: https://www.hivemq.com/docs/hivemq/4.x/user-guide/

[4] Node.js Foundation, *Node.js Documentation v18.x*, 2024. [Online]. Available: https://nodejs.org/docs/latest-v18.x/api/

[5] Express.js Team, *Express.js Guide and API Reference*, 2024. [Online]. Available: https://expressjs.com/en/guide/

[6] Socket.IO, *Socket.IO Documentation v4*, 2024. [Online]. Available: https://socket.io/docs/v4/

[7] MySQL AB, *MySQL 8.0 Reference Manual*, Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[8] OpenAPI Initiative, *OpenAPI Specification 3.0.3*, 2021. [Online]. Available: https://spec.openapis.org/oas/v3.0.3

[9] Chart.js Contributors, *Chart.js Documentation*, 2024. [Online]. Available: https://www.chartjs.org/docs/latest/

[10] Swagger, *Swagger UI Documentation*, SmartBear Software, 2024. [Online]. Available: https://swagger.io/tools/swagger-ui/